TP 1 — Sentimiento de tweets sobre IA

Decisiones del pipeline y su justificación · Ciencia de Datos, UTN FRBA · 2.º cuatrimestre 2026

Este documento reúne, etapa por etapa, **(1)** las decisiones que tomamos y por qué, y **(2)** las decisiones que habría tomado Claude de resolverlo por su cuenta, marcando dónde difieren. Todas las cifras salen de experimentos propios sobre *ia_tweets.csv* (validación cruzada estratificada, macro-F1). Documento elaborado con asistencia de Claude (IA generativa); conviene declararlo en la sección final del notebook, como pide la consigna.

Punto de partida y resultado

Dataset	4038 tweets en español sobre IA, 14 columnas, sin nulos. Objetivo: predecir **polarity** (N / NEU / P).
Después de limpiar	3981 tweets. Clases: NEU 67,3 % · P 17,8 % · N 14,9 % (desbalanceado).
Pipeline final	Limpieza de texto → TF-IDF + 5 features de expresividad → SVM lineal (class_weight='balanced') → Grid Search sobre C.
Resultado en test	Macro-F1 **0,586** · weighted-F1 0,698 · accuracy 0,705 (una sola evaluación sobre 797 tweets nunca vistos). Baseline «siempre NEU»: accuracy 0,674 pero macro-F1 0,268.

Parte 1 · Decisiones que tomamos y su justificación

Cuando una decisión surgió de una discusión (por ejemplo la métrica), se indica. Una advertencia honesta: en las pruebas A/B, salvo conservar los emojis, el resto de las decisiones de texto quedó **dentro del ruido** entre folds (±0,01–0,02), así que su justificación es principalmente teórica y se apoya en la evidencia exploratoria.

Etapa	Decisión	Justificación
Columnas	Eliminar **ID**, **user_is_translator** y **user_protected**.	ID coincide con el índice (no informa nada). Las otras dos valen 0 en todas las filas: una columna constante no puede distinguir clases.
Duplicados	Eliminar duplicados exactos del texto.	57 filas repetidas (11 textos distintos), sin etiquetas contradictorias: 4038 → 3981. Evita que el mismo tweet caiga en train y test (*data leakage*) e infle las métricas.
Desbalance	**class_weight='balanced'** + división y folds **estratificados**.	La penalización de cada error pasa a ser $n/(k \cdot n_c)$: equivocarse en una clase chica cuesta más. No altera los datos. Undersampling descartaría 2205 tweets (55 %); oversampling duplica ejemplos y puede filtrarse entre folds. Evidencia: macro-F1 0,426 → 0,567 y recall de N 0,10 → 0,56. Estratificar mantiene 67/18/15 % en train, test y cada fold.
Métrica	**Macro-F1** como principal; weighted-F1 y recall/F1 por clase como secundarias.	Elegiste Weighted-F1 al principio y lo discutimos. Pesó la *resolución*: entre un modelo que casi ignora N y P y uno que los detecta, el weighted se mueve 0,023 (≈1,5 veces el ruido) y el macro 0,141 (≈8 veces). «Siempre NEU» da 0,542 de weighted-F1 pero 0,268 de macro. Además lo informativo para la organización son las opiniones P y N.
Outliers	Transformación **log(1+x)** en variables de cola larga.	favorite_count tiene 52,5 % de ceros y retweet_count 84,6 %; la regla del IQR queda degenerada. El log conserva todos los tweets, mantiene el 0 y comprime los virales. Se usa en el EDA y en las features de conteo, seguido de StandardScaler (necesario para SVM).
Variables	**Solo texto**: sin metadatos de usuario ni interacción, más features derivadas del texto (expresividad: n.º de «!», «?», proporción de mayúsculas, alargues como «holaaa», risas «jaja»).	Agregar columnas numéricas al texto no mejoró (macro-F1 0,567 → 0,568 → 0,569, ruido). Solas tienen algo de señal (0,46) pero puede ser una «huella» del usuario y no del sentimiento. Las features de expresividad salen del propio tweet y aportan poco (~+0,01, cerca del ruido).
URLs	Eliminarlas.	Cada link distinto sería un token único, con una columna casi vacía por link: ruido sin poder de generalización.
@menciones	Eliminarlas.	Son identificadores de usuario de alta cardinalidad (52,8 % de los tweets tiene alguna), y el modelo no debería aprender de nombres propios.

Etapa	Decisión	Justificación
Hashtags	Conservar la palabra y **separar CamelCase** (#InteligenciaArtificial → «inteligencia artificial»).	Tienen contenido semántico (aparecen en 15,4 % de los tweets y 28,8 % de los P). Separarlos los reconecta con el vocabulario del resto del texto.
Emojis	**Conservarlos como tokens.**	Aparecen en 22,6 % de los tweets (29 % de los P) y cargan emoción. El tokenizador por defecto los descarta. Fue la única decisión de texto con mejora clara (+0,016).
Stopwords	**No eliminarlas.**	Las negaciones («no», «nunca», «sin») cambian el sentimiento; eliminarlas borra justo lo que importa. TF-IDF ya baja el peso de las palabras comunes.
Raíces	**Stemming** (Snowball en español).	Reduce el vocabulario (menos columnas, menos overfitting). La lematización necesita spaCy, que queda fuera de lo visto en clase. Ojo: PorterStemmer de las diapositivas es solo para inglés.
N-gramas	Solo palabras sueltas (unigramas).	Los bigramas multiplican las columnas con pocos datos y ya hay overfitting. Se deja como mejora futura.
Vectorización	**TF-IDF** (min_df=2), ajustado solo con train.	Baja el peso de palabras presentes en casi todos los tweets («ia», «inteligencia») y sube el de las distintivas. Ajustarlo solo con train evita *data leakage*.
Modelo	**SVM lineal** (LinearSVC).	Elegiste limitarte a lo visto en clase: la SVM está en la Clase 6 y la consigna la nombra. Rinde igual que el mejor (0,595), el kernel lineal es el que recomienda la diapositiva con muchas features, y escala mucho mejor que el RBF (0,6 s vs 3,0 s por entrenamiento). Su hiperparámetro principal es C.
Validación y búsqueda	División 80/20 estratificada; **K-fold estratificado (5)** y **Grid Search** sobre C solo en el 80 %.	El test se toca **una sola vez**: si se usara para elegir C, la estimación final quedaría optimista (selection bias). El Grid Search es exhaustivo y la grilla es chica (9 valores de C), así que no hace falta Randomized Search.

Resultados que respaldan las decisiones

Comparación de modelos (5-fold estratificado sobre los 3981 tweets, macro-F1; la brecha train/test indica overfitting):

Modelo	Macro-F1 test	Weighted-F1	Macro-F1 train	Comentario
Gaussian Naive Bayes	0,436 ± 0,011	0,520	0,744	Asume features normales e independientes; mal ajuste con texto
Árbol de decisión (sin límite)	0,473	0,598	0,998	Memoriza el train
Árbol (max_depth=10)	0,474	0,592	0,599	Limitar la profundidad no ayuda
Random Forest (150 árboles)	0,546 ± 0,016	0,676	0,996	Overfitting fuerte con miles de columnas casi vacías
SVM kernel RBF	0,554 ± 0,024	0,633	0,727	Más lenta y con más variabilidad
Regresión Logística	0,594 ± 0,011	0,671	0,834	Empata con la SVM lineal; menor brecha train/test
SVM lineal	**0,595 ± 0,013**	0,700	0,968	Elegido. Subir max_iter evita el warning de convergencia

Todos usan class_weight='balanced' salvo Naive Bayes, que no lo admite (se fijaron priors iguales). El Naive Bayes multinomial, que sí es el natural para conteos de palabras, dio 0,509.

Grid Search sobre C (SVM lineal, 5-fold estratificado dentro del 80 % de train):

C	0,001	0,01	0,03	0,1	0,3	1 (defecto)	3	10	100
Train	0,362	0,372	0,557	0,818	0,925	0,978	0,991	0,994	0,994
Validación	0,359	0,366	0,443	0,550	**0,583**	0,581	0,571	0,560	0,552

C bajo = **underfitting** (train y validación igual de malos). C alto = **overfitting** (train ≈ 0,99, validación baja). El mejor es C = 0,3, pero su diferencia con el valor por defecto (0,583 vs 0,581) es menor que el desvío entre folds (±0,03): el Grid Search confirmó que el defecto ya era casi óptimo, y su valor es mostrar la curva completa (curva de validación, Clase 5).

Evaluación final en test (797 tweets, una sola vez):

Clase	Precision	Recall	F1	Soporte
N (negativo)	0,478	0,364	0,413	118
NEU (neutral)	0,779	0,825	0,801	537
P (positivo)	0,551	0,535	0,543	142

Real \ Predicho	N	NEU	P
N	43	67	8
NEU	40	443	54
P	7	59	76

La clase negativa es la más difícil: solo se detectan 43 de 118 y la mayoría de los errores son N confundidos con NEU (67), no con P. Como N tiene apenas 118 tweets en test, las métricas tienen un margen de error de varios puntos, y conviene decirlo en la presentación.

Parte 2 · Decisiones que habría tomado yo (Claude)

«Coincido» significa que, con la evidencia medida, habría elegido lo mismo. Las diferencias reales son pocas y están **resaltadas en amarillo**. En las decisiones de texto, donde las pruebas dieron empate técnico, ambas opciones son defendibles; lo importante es que cada integrante pueda explicar el porqué.

Etapa	Mi decisión	¿Coincide?	Razonamiento / diferencia
Columnas y duplicados	Eliminar ID, columnas constantes y duplicados exactos.	Coincido	Una columna constante no aporta y los duplicados exactos son riesgo de leakage. Los quitaría antes de dividir en train/test.
Desbalance	class_weight='balanced' + estratificación.	Coincido	Es lo más barato que mejora el recall de N y P sin tocar los datos. Mostraría en el notebook el baseline «siempre NEU» para explicar la paradoja del accuracy.
Métrica	Macro-F1 como principal.	Coincido (al final)	Partimos de posturas distintas: vos Weighted-F1, yo Macro-F1. Tras discutir con la evidencia de la resolución coincidimos. Reportaría igual weighted-F1, recall y F1 por clase.
Outliers	log(1+x); como segunda opción, recortar en el percentil 99.	Coincido	Descartaría eliminar filas (son datos reales). Como el log comprime pero no elimina extremos, escalaría después con StandardScaler.
Variables	Solo texto + features de expresividad.	Coincido, con una salvedad	Repetiría en el notebook el experimento «texto vs texto + numéricas» y lo mostraría como justificación. Haría además una validación agrupada por usuario para chequear si los metadatos filtran la «huella» del usuario.
Texto	URLs y menciones fuera; hashtags separados en CamelCase; emojis como tokens; stopwords conservadas.	Coincido	Salvo emojis, las alternativas dan resultados equivalentes, así que priorizo la lógica lingüística (negaciones, contenido de los hashtags). Una alternativa a probar: conservar un indicador binario «tiene URL» en vez de borrar toda la información.
Raíces y n-gramas	Stemming Snowball; solo unigramas.	Coincido	Con ~4000 tweets y overfitting visible, conviene vocabulario chico. Lematización con spaCy y bigramas quedan como mejoras.
Vectorización	TF-IDF con min_df=2, ajustado solo en train.	Coincido	Meter el vectorizador dentro del Pipeline garantiza que cada fold lo reajuste sin leakage.
Modelo a afinar	**Regresión Logística**, o mejor aún las dos (logística y SVM lineal).	**Difiero**	Empatan en rendimiento (0,594 vs 0,595). La logística tiene menos brecha train/test (0,834 vs 0,968), una perilla principal, es interpretable (se ven las palabras que empujan hacia N o P) y da probabilidades. La consigna la lista, aunque las diapositivas no. Elegir SVM lineal para quedarse en lo visto en clase es una decisión defendible; solo conviene que quede explicado.
Otros modelos	Una exploración corta de cada modelo (p. ej. max_depth del árbol, n_estimators y max_depth del Random Forest, var_smoothing del Naive Bayes).	**Difiero / pendiente**	La consigna pide «investiguen qué hiperparámetros pueden ajustarse en cada modelo y experimenten». Hoy solo afinamos la SVM; dejar los demás como baselines sin explorar puede costar puntos. Es una decisión que todavía no cerramos.
Validación y búsqueda	K-fold estratificado de 5 + Grid Search sobre C; test una sola vez.	Coincido	Además reportaría la validación agrupada por usuario como chequeo de robustez: en nuestras pruebas baja todos los puntajes unos 0,016, lo que da una idea de cuánto optimismo tiene la estimación con folds comunes.
Cierre	Análisis de errores sobre tweets mal clasificados + curva de validación en el notebook.	Adicional	Mirar ejemplos de N predichos como NEU explica el techo del modelo (ironía, sarcasmo) y alimenta la sección de conclusiones.

Lo que falta para entregar

Pendiente	Detalle
Hiperparámetros del resto de los modelos	Decidir si se hace una exploración corta de cada uno o se dejan como baselines justificados.
Análisis de errores	Mirar tweets mal clasificados, sobre todo N predichos como NEU.
Conclusiones y mejoras futuras	Marcado de negaciones, lematización con spaCy, embeddings, bigramas, validación agrupada por usuario, más datos de la clase N.
Notebook (.ipynb)	EDA, preprocesamiento, features, entrenamiento, evaluación y conclusiones, con cada decisión explicada. Debe correr completo con «Run All» y cerrar con la sección que declara dónde y para qué se usó IA generativa.
Presentación (20 min)	Enfoque, hallazgos, decisiones, qué funcionó mejor y qué se probó, mejoras futuras. Los docentes pueden preguntar por cualquier parte del notebook, incluso las que no escribieron los integrantes: conviene que todos entiendan cada decisión.